

ENTERPRISE AI RESEARCH AGENT

Banking — Technical Documentation

Modus Enterprise AI Build Challenge — Assignment 9

Project	Enterprise AI Research Agent — Banking
Purpose	Structured, traceable AI-assisted enterprise research
Primary UI	Streamlit
API / Backend	FastAPI
AI Layer	Groq LLM
Research	DuckDuckGo search + page text extraction
Persistence	SQLite
Repository	github.com/irindaisyashok/banking-ai-research-agent

Document purpose

This document explains the system design, architecture, research workflow, data model, API surface, deployment approach, setup procedure, limitations, and future scalability plan for the Banking AI Research Agent.

Repository: <https://github.com/irindaisyashok/banking-ai-research-agent>

1. Project Overview

The Enterprise AI Research Agent is an AI application designed to conduct structured and traceable research on how AI is transforming banking. Instead of treating research as a single chatbot response, the application decomposes a question into explicit stages, stores research artifacts, compares evidence, detects contradictions, and produces a conclusion linked back to supporting findings and source URLs.

The design emphasizes auditability and reuse. Research results are persisted in SQLite, allowing users to browse previously researched topics and questions without re-running the pipeline.

Key objectives

- Provide a reusable research workflow for banking and enterprise-AI questions.
- Separate user interface, API, orchestration, AI reasoning, external research, and persistence responsibilities.
- Persist sources, findings, contradictions, conclusions, and evidence links.
- Make conclusions traceable to specific findings and source URLs.
- Accept new, previously unseen questions through the same pipeline.
- Provide an interactive UI and a Swagger/OpenAPI interface for technical evaluation.

2. Functional Requirements

Requirement	Implementation
New research question	Streamlit UI submits a question through FastAPI; the same pipeline handles the request.
Topic-level research	A broad topic can be decomposed into multiple sub-questions and researched as a batch.
Web research	Search results and page text are collected through the external research client.
Finding extraction	The LLM extracts structured findings from collected source material.
Finding classification	Findings are classified into Law, Guidance, Standard, Research, Vendor, or General.
Evidence comparison	Findings are compared to identify agreement and conflicting evidence.
Contradiction detection	Potential contradictions are persisted as links between two findings.
Conclusion generation	A conclusion is generated and linked to supporting findings.
Knowledge-base browsing	Existing topics/questions can be browsed and inspected from SQLite-backed data.

3. System Architecture

The application uses a layered architecture. The Streamlit UI communicates with the FastAPI application layer. The API invokes the orchestration pipeline, which coordinates LLM reasoning, external research, and database persistence.

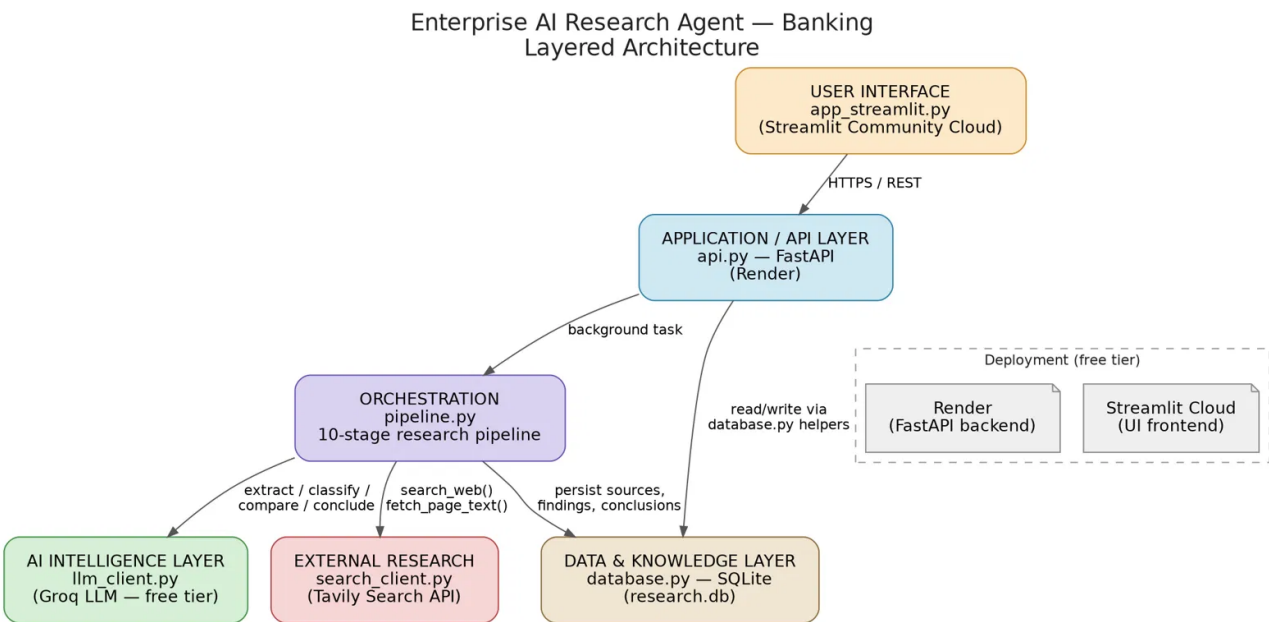


Figure 1. Layered architecture and deployment view.

3.1 Layer responsibilities

Module	Layer	Responsibility
app_streamlit.py	User Interface	Renders the application, collects input, displays pipeline progress, conclusions, evidence.
api.py	Application / API	FastAPI endpoints, request handling, validation, and access to the research workflow.
pipeline.py	Orchestration	Runs the 10-stage research workflow and coordinates research, LLM processing, comparison.
llm_client.py	AI Intelligence	Centralizes LLM reasoning calls through Groq.
search_client.py	External Research	Performs web search and page text extraction.
database.py	Data & Knowledge	Defines SQLite persistence and database helper operations.

3.2 Data and control flow

- The user submits a banking/AI research question through Streamlit.
- Streamlit calls the FastAPI backend over HTTP.
- The API invokes the research orchestration pipeline.
- The pipeline searches external sources and fetches page content.
- The LLM extracts and classifies findings from the collected material.
- Findings are compared and contradictions are detected.
- The generated conclusion is persisted together with evidence links.
- The UI retrieves the persisted result and displays the conclusion and supporting evidence.

4. Research Pipeline

The core workflow is implemented in `pipeline.py` as a reusable, question-independent 10-stage process. This supports the surprise-record evaluation because a new question does not require question-specific code.

Stage	Activity	Output / Purpose
1	Define research questions	Establish the research question or sub-question.
2	Search sources	Find potentially relevant web sources.
3	Collect information	Fetch and extract usable page text.
4	Store sources	Persist source URL, title, type, snippet/content metadata.
5	Extract findings	Use the LLM to convert source material into research findings.
6	Classify findings	Categorize evidence by source type.
7	Compare evidence	Evaluate findings for agreement and differences.
8	Detect contradictions	Identify conflicting findings and store the relationship.
9	Generate conclusions	Produce a synthesized conclusion from the evidence.
10	Maintain traceability	Link conclusions to findings and findings to sources.

4.1 Topic research

The application can accept a broad topic and a configurable number of sub-questions. The topic is decomposed, and each sub-question is sent through the same research pipeline.

4.2 Surprise-question workflow

A new question can be submitted through the UI or `POST /questions`. The application does not depend on preconfigured question text; the same workflow is executed for new questions.

5. AI / LLM Design

The AI reasoning layer is isolated in `llm_client.py`. This keeps model-specific request logic separate from the research orchestration and UI.

- LLM usage is concentrated in the AI intelligence layer rather than being embedded throughout the application.
- The LLM supports extraction, classification, evidence comparison, contradiction analysis, and conclusion generation.
- The application uses Groq's free-tier LLM service as configured by the project.
- The model provider can be replaced without redesigning the rest of the application; the project documentation identifies Ollama as a possible local-model fallback.

Traceability principle

Conclusions are linked to evidence through the `evidence_links` relationship. Findings retain their relationship to source records, including URLs. This allows a reviewer to inspect which evidence contributed to a conclusion rather than receiving an unsupported generated response.

6. Database and Data Model

SQLite is used as the persistent knowledge store. The model separates research topics, questions, sources, findings, contradictions, conclusions, and evidence relationships.

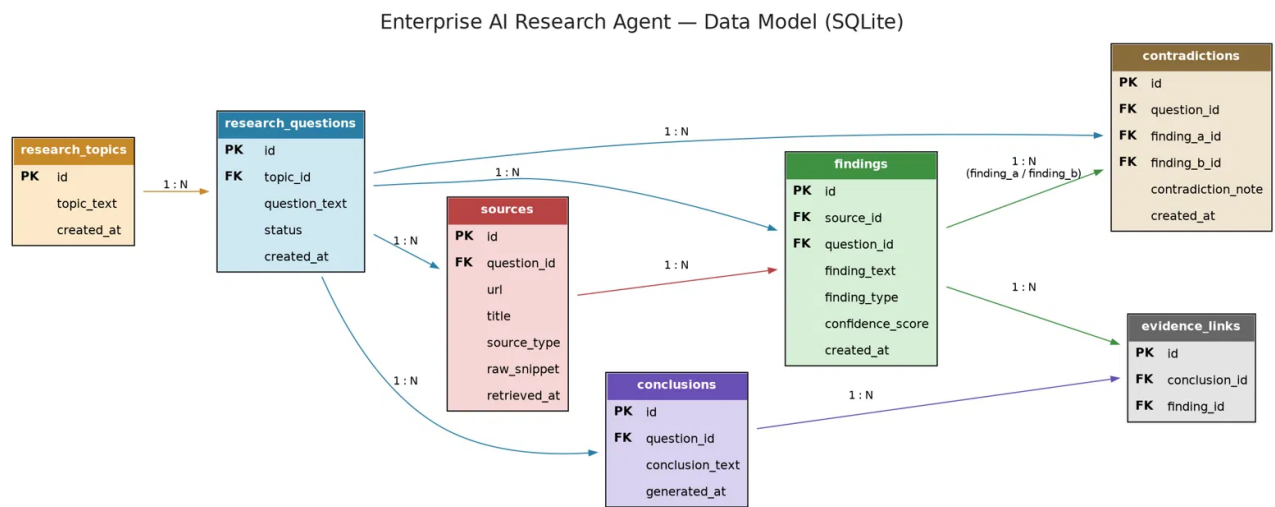


Figure 2. SQLite enterprise research data model.

Table	Purpose	Key relationships
research_topics	Stores high-level research topics.	One topic can contain many research questions.
research_questions	Stores individual research questions and status.	Belongs to a topic; has sources, findings, and conclusion.
sources	Stores discovered research sources and metadata.	Belongs to a question; can produce findings.
findings	Stores extracted evidence and confidence information.	Belongs to a source and question.
contradictions	Stores conflicting evidence relationships.	Links finding A and finding B for a question.
conclusions	Stores synthesized conclusions.	Belongs to a question.
evidence_links	Associates conclusions with supporting findings.	Many-to-many bridge between conclusions and findings.

7. API Documentation

The FastAPI service exposes an interactive Swagger UI at /docs when the backend is running. The API provides programmatic access to research creation and persisted evidence.

Method	Endpoint	Purpose
POST	/questions	Submit a new research question and run the research workflow.
POST	/topics	Submit a broader topic for decomposition and research.
GET	/topics	List previously researched topics.
GET	/questions/{id}/sources	Retrieve sources associated with a question.
GET	/questions/{id}/findings	Retrieve extracted findings.
GET	/questions/{id}/contradictions	Retrieve detected contradictions.
GET	/questions/{id}/conclusion	Retrieve conclusion and linked supporting evidence.

Example request

```
curl -X POST http://localhost:8000/questions \ -H "Content-Type: application/json" \ -d  
'{"question_text":"How are banks using AI for trade finance document checking?"}'
```

For the final submission, provide the deployed FastAPI /docs URL as the optional API Documentation link after verifying public access.

8. Installation and Local Setup

1. Use Python 3.10 or newer.
2. Create and activate a virtual environment.
3. Install dependencies from requirements.txt.
4. Create a .env file from .env.example and configure GROQ_API_KEY.
5. Optionally run seed_data.py to populate the knowledge base.
6. Start FastAPI with uvicorn api:app --reload --port 8000.
7. Start Streamlit with streamlit run app_streamlit.py.

Example commands

```
python -m venv venv # Windows venv\Scripts\activate pip install -r requirements.txt uvicorn api:app --reload --port 8000 streamlit run app_streamlit.py
```

The project documentation describes Streamlit as the UI deployment and Render as the FastAPI backend deployment. Environment secrets such as the Groq API key should be configured through the deployment platform rather than committed to source control.

9. Security and Reliability Considerations

- API credentials should be stored as environment variables and never committed to GitHub.
- External web content should be treated as untrusted input; fetched text should be constrained and validated before downstream processing.
- LLM-generated conclusions should be presented with source-backed evidence and should not be treated as authoritative financial or regulatory advice.
- API request validation should reject malformed inputs before triggering expensive research work.
- For production use, authentication, rate limiting, structured logging, monitoring, retries, and stronger database controls should be added.

10. Testing and Evaluation Approach

The system supports both a normal demonstration flow and an evaluator-driven surprise question. The key functional test is whether a previously unseen question can pass through the same pipeline and produce a persisted, traceable result.

Test area	Expected behavior
New question	Accept a new question without hard-coded question-specific logic.
Source retrieval	Return usable source records associated with the question.
Finding extraction	Persist extracted findings with source/question relationships.
Contradiction detection	Persist conflicting finding pairs when detected.
Conclusion	Return a synthesized conclusion with supporting evidence.
Persistence	Previously researched records remain available after application restart.
Knowledge-base browsing	Existing records are retrieved from SQLite rather than recomputed.
API	Swagger/OpenAPI endpoints can be exercised independently.

11. Scalability and Production Roadmap

The current implementation is intentionally lightweight and suitable for the challenge scope. Scaling to hundreds or thousands of concurrent research processes would require additional infrastructure.

- Replace synchronous in-request execution with a queue-based asynchronous worker model such as Celery or RQ.
- Move from SQLite to PostgreSQL for concurrent multi-user writes.
- Introduce a vector store over findings for semantic evidence retrieval and scalable comparison.
- Cache and deduplicate previously fetched sources to reduce repeated network calls.
- Add observability: structured logs, metrics, tracing, error monitoring, and job status tracking.
- Add authentication and role-based access control for enterprise use.

12. Current Limitations

- SQLite is not intended for high-concurrency production workloads.
- The research workflow depends on availability and quality of external web search and page content.
- LLM outputs can contain uncertainty or errors; source-backed evidence and confidence information are therefore important.
- The current free-tier deployment approach is suitable for demonstration but is not a complete enterprise production architecture.
- No financial decision, credit decision, investment recommendation, or regulatory conclusion should be made solely from generated output.

13. Technology Stack

Component	Technology
Language	Python
Frontend	Streamlit
Backend/API	FastAPI
AI/LLM	Groq API / configured free-tier LLM
Web research	DuckDuckGo search + page text extraction
Database	SQLite
HTTP / parsing	Requests, BeautifulSoup4
Deployment	Streamlit Cloud for UI; Render for FastAPI backend
API interface	OpenAPI / Swagger UI via FastAPI

14. Repository Structure

```
banking-ai-research-agent/ ■■■ app_streamlit.py # Streamlit user interface ■■■ api.py # FastAPI application and endpoints ■■■ pipeline.py # 10-stage research orchestration ■■■ llm_client.py # LLM integration ■■■ search_client.py # Web search and page extraction ■■■ database.py # SQLite schema and persistence ■■■ seed_data.py # Initial knowledge-base seeding ■■■ requirements.txt # Python dependencies ■■■ runtime.txt # Runtime configuration ■■■ .env.example # Environment variable template ■■■ README.md # Project overview and setup
```

15. Challenge Alignment

Challenge expectation	How this implementation addresses it
AI-powered application	LLM-driven research extraction, classification, comparison, contradiction detection, and conclusions
Not a single prompt / chatbot wrapper	A discrete 10-stage persisted pipeline is used.
Persistent data	SQLite stores topics, questions, sources, findings, contradictions, conclusions, and evidence links
Reusable workflow	New questions execute through the same question-independent pipeline.
Traceability	Conclusions link to findings, and findings link to sources.
Technical accessibility	FastAPI Swagger interface and Streamlit UI are both available.
Free/open-source challenge scope	The repository documents use of free-tier/open-source components.

16. Conclusion

The Banking Enterprise AI Research Agent demonstrates a modular approach to AI-assisted enterprise research. Its main technical strength is the combination of LLM reasoning with persistent, source-linked evidence rather than relying on an untraceable single response. The layered architecture keeps UI, API, orchestration, AI, research, and data responsibilities separated, while the SQLite knowledge base makes previous research reusable.

For production deployment, the next architectural step is asynchronous job processing, a production-grade relational database, scalable evidence retrieval, source caching, authentication, monitoring, and stronger security controls.

Repository: <https://github.com/irindaisyashok/banking-ai-research-agent>

Submission note: Add the live application URL and deployed Swagger URL to the final application form after verifying that both deployments are publicly accessible.